



Efficient digital signatures from RSA without random oracles

Jae Hong Seo

Department of Mathematics & Research Institute for Natural Sciences, Hanyang University, Seoul 04763, Republic of Korea



ARTICLE INFO

Article history:

Received 6 September 2018

Revised 5 August 2019

Accepted 30 September 2019

Available online 1 October 2019

Keywords:

RSA

Digital signatures

Prime-number generation

Standard model

ABSTRACT

Improving efficiency of digital signature scheme is important since digital signature scheme is a core building block for many privacy protocols. There are some proposals regarding efficient digital signatures whose security arguments are guaranteed by the standard assumption such as RSA assumption. Although several RSA-based digital signature schemes achieve a short signature size, many of them essentially rely on random oracle heuristics. In 2009, Hohenberger and Water proposed an excellent approach to the design of a short RSA-based signature scheme *without* random oracles (CRYPTO 2009). However, their scheme requires signers to execute an expensive prime-number generation several times, and leaves the reduction in signing and verifying costs as important open problems.

In this paper, we propose an efficient digital signature scheme from the above category. That is, we propose a short RSA signature scheme in the standard model, which requires less prime-number generations than those in the previous best scheme of Böhl, Hofheinz, Jager, Koch, and Striecks (Journal of Cryptology 2015). More precisely, the BHJKS scheme requires signers to generate $O(\log \lambda)$ prime-numbers for each signature; however, our scheme requires almost a constant time (e.g., $\log \log \lambda$) of prime-number generation in the signing algorithm, where λ is the security parameter.

© 2019 Elsevier Inc. All rights reserved.

1. Introduction

A digital signature scheme is a fundamental primitive of modern cryptography. Although the concept of a digital signature scheme is simple, there have been a large number of studies on diverse functionalities to improve the efficiency and/or to reinforce the security of digital signatures. Because the original digital signature can be an important building block for crypto protocols, as well as a good baseline to build more functional signatures, it is important to design efficient signature schemes whose security is proven under well-understood assumptions. For example, redactable signature scheme is a useful primitive in some scenarios such that authenticity of anonymized data is required [7,20,24]. There exist efficient generic constructions for redactable signature scheme based on a normal digital signature scheme [20,24]. Furthermore, homomorphic signature scheme [5,6,20], which is a very useful primitive for network coding [1], can be *almost* generically built on the basis of a normal signature scheme with mild properties satisfied by many of known signature schemes [13]. Therefore, designing normal signature scheme with improved efficiency under reliable assumption is very important.

In this study, we consider *short* digital signature schemes based on the hardness of the *standard* hard problem, particularly *without* random oracle heuristics. Here, “short” means constant in the security parameter. Although there are several excellent approaches to designing digital signature schemes, we do not consider some of them in this study because they do not successfully create short signatures. For example, there have been significant efforts to design a digital signature

E-mail address: jaehongseo@hanyang.ac.kr

scheme from any one-way functions [3,22,23,26] or a tree-based approach [3,9,11]. Although such studies are theoretically important, both approaches are regarded as less practical in the sense of the signature size.¹ If we use random oracle heuristics [2] or strong complexity assumption, such as a strong RSA [8,10,12,14,17], we can then construct a more efficient digital signature scheme. Such approaches provide quite reliable security arguments because there are no real-world attacks against such stronger assumptions. However, it is clear that these are stronger assumptions than well-understood/analyzed standard complexity assumptions such as RSA and computational Diffie–Hellman in the standard model. In this study, we focus on a standard model RSA signature scheme with constant size signatures.

Hohenberger and Waters proposed the first short and stateless RSA signature scheme in the standard model [19]. Their scheme is quite efficient in terms of the public key and signature sizes but requires signers to conduct an expensive prime-number generation $O(\lambda)$ times, where λ is the security parameter. Subsequently, Böhl, Hofheinz, Jäger, Koch, and Striecks improved the Hohenberger–Waters signatures in terms of the number of required prime-number generations in the signing algorithm to be $O(\log \lambda)$ through the use of a new proof technique called “confined-guessing.” In this study, we reduce the required number of prime-number generations to be almost constant, or more precisely, $\omega(1)$ (e.g., $\log \log \lambda$). Our scheme is a *tag-based* signature scheme such as Böhl et al.’s scheme; however, we use a different shape of the tag vectors, and more importantly, a “prefix-guessing” proof technique based on the generalized birthday lemma [16,27]. As a direct application, if we combine our result with Nojima et al.’s transformation to a redactable signature scheme [24], then we obtain an efficient redactable signature from the RSA assumption in the standard model.

Outline. In the next section, we provide useful preliminaries to read the remaining sections. In Section 3, we present our digital signature scheme, and analyze its security in Section 4. We wrap up in Section 6

2. Preliminaries

In this section, we describe the notations used in this paper, standard definitions of a digital signature scheme, and the backgrounds regarding number theory and number-theoretic hardness assumption.

2.1. Notations

Let us begin with useful notations used throughout the paper. Like many other public key cryptography literatures, we will use λ to denote the security parameter. For the two integers $a \leq b$, $[a, b]$ denotes a set of continuous integers between a and b , including both a and b . When randomly choosing an element a from a set S , we use notation $a \xleftarrow{\$} S$. For a probabilistic algorithm A , we use notation $A(x) \rightarrow y$ to denote that A takes x as input, and then outputs y with its own random tape. If A terminates within polynomial time in λ , then we say that A is a probabilistic polynomial-time (PPT) algorithm. Finally, we use a standard concept of negligible function; a function $\text{neg}(\lambda) : \mathbb{N} \rightarrow \mathbb{R}$ is a negligible function if for $\forall$ positive polynomial $\text{poly}(\cdot)$, there exists a positive integer n_{poly} such that for every $\lambda > n_{\text{poly}}$, an inequality $|\text{neg}(\lambda)| < \frac{1}{\text{poly}(\lambda)}$ holds.

2.2. Digital signature scheme

We briefly review the standard definitions of digital signature scheme. A digital signature scheme with message space $\mathcal{M}$ consists of three PPT algorithms, *Setup*, *Sign*, and *Verify* as follows. *Sign*, and *Verify*.

Setup(λ): It takes the security parameter λ and outputs (vk, sk) .

Sign(M, sk): It takes sk and a message $M \in \mathcal{M}$, and outputs a signature σ .

Verify(vk, M, σ): It takes $vk, M \in \mathcal{M}$, and σ as inputs, and returns either accept or reject.

We require every digital signature scheme $\mathcal{S} = \{\text{Setup}, \text{Sign}, \text{Verify}\}$ to satisfy the following correctness condition. For arbitrary $m \in \mathcal{M}$,

$$\Pr[\text{Setup}(\lambda) \rightarrow (vk, sk), \text{Verify}(vk, m, \text{Sign}(m, sk)) \rightarrow 1] = 1.$$

We show that our signature scheme satisfies the standard security notion *existential unforgeability with respect to chosen-message attacks* (EU-CMA) [15]. To achieve EU-CMA security without relying on random oracle heuristic, we basically follow the approach used by all previous RSA signature schemes [4,16,19]. That is, it is sufficient to prove that the proposed signature scheme achieves a slightly weaker security notion called *existential unforgeability with respect to weak chosen-message attacks* (EU-weakCMA) because there exists a well-known generic transformation from an EU-weakCMA secure scheme to an EU-CMA secure scheme using chameleon hashes [21]. Such the transformation is quite efficient and there exist efficient chameleon hashes based on the RSA assumption in the standard model [18]. Therefore, if we combine our EU-weakCMA scheme with such RSA-based chameleon hashes, we will eventually obtain an EU-CMA secure signature scheme from the RSA assumption in the standard model. Note that all previous standard model RSA signature schemes follow this approach, that is, the authors first propose an EU-weakCMA secure scheme and then combine it with RSA-based chameleon hashes.

¹ Nevertheless, recently these approaches are brought back into the spotlight, as a candidate methodology for designing post-quantum signatures [3].

The definitions of EU-CMA and EU-weakCMA can be found in previous studies [4,16,19,27]. We recall the definitions in [27]; for the signature scheme $\mathcal{S} = (\text{Setup}, \text{Sign}, \text{Verify})$ and adversary $\mathcal{A}$, we first define the experiment $\text{Exp}_{\mathcal{S}, \mathcal{A}}^{\text{EU-CMA}}$ as follows.

$\text{Exp}_{\mathcal{S}, \mathcal{A}}^{\text{EU-CMA}}(\lambda)$
 $(vk, sk) \leftarrow \text{Setup}(\lambda); (M, \sigma) \leftarrow \mathcal{A}^{\text{Sign}(\cdot)}(vk);$
 Let $List$ be a set of all messages queries;
 If $M \notin List$ and $\text{Verify}(vk, M, \sigma) = 1$, output 1.
 Otherwise, output 0.

In the above experiment, $\mathcal{A}^{\text{Sign}(\cdot)}$ denotes that $\mathcal{A}$ can adaptively access to a signing oracle $\text{Sign}(\cdot)$; that is, whenever $\mathcal{A}$ issues a message, the oracle outputs a corresponding signature.

Next, we define another experiment $\text{Exp}_{\mathcal{S}, \mathcal{A}=(\mathcal{A}_1, \mathcal{A}_2)}^{\text{EU-weakCMA}}$, which is basically the same as $\text{Exp}_{\mathcal{S}, \mathcal{A}}^{\text{EU-CMA}}$, except that the adversary $\mathcal{A}$ is required to send all signing queries before she sees a verification key. More precisely, in this experiment, $\mathcal{A}$ consists of two distinct algorithms $\mathcal{A}_1$ and $\mathcal{A}_2$, where $\mathcal{A}_1$ and $\mathcal{A}_2$ can share something through a state information denoted by st , and $\mathcal{A}_1$ should output a set $List$ of messages at the very beginning time. The concrete description of this experiment is given below.

$\text{Exp}_{\mathcal{S}, \mathcal{A}=(\mathcal{A}_1, \mathcal{A}_2)}^{\text{EU-weakCMA}}(\lambda)$
 $(List, st) \leftarrow \mathcal{A}_1; (vk, sk) \leftarrow \text{Setup}(\lambda);$
 For $\forall M_i \in List, \sigma_i \leftarrow \text{Sign}(M_i, sk);$
 $(M, \sigma) \leftarrow \mathcal{A}_2(vk, \sigma_1, \dots, \sigma_q, st);$
 If $M \notin List$ and $\text{Verify}(vk, M, \sigma) = 1$, output 1.
 Otherwise, output 0.

Now, we define the adversarial advantages of two above experiments as follows.

$$\text{Adv}_{\mathcal{S}, \mathcal{A}}^{\text{EU-CMA}}(\lambda) = \Pr [\text{Exp}_{\mathcal{S}, \mathcal{A}}^{\text{EU-CMA}}(\lambda) = 1]$$

$$\text{and } \text{Adv}_{\mathcal{S}, \mathcal{A}}^{\text{EU-weakCMA}}(\lambda) = \Pr [\text{Exp}_{\mathcal{S}, \mathcal{A}}^{\text{EU-weakCMA}}(\lambda) = 1].$$

Using the above notions, we define EU-CMA and EU-weakCMA as follows.

Definition 1. Let $\mathcal{S}$ be a signature scheme. We say that $\mathcal{S}$ is EU-CMA or EU-weakCMA secure if, for any PPT adversary, $\mathcal{A}$, $\text{Adv}_{\mathcal{S}, \mathcal{A}}^{\text{EU-CMA}}(\lambda)$ or $\text{Adv}_{\mathcal{S}, \mathcal{A}}^{\text{EU-weakCMA}}(\lambda)$ is a negligible function in λ , respectively.

2.3. Generic transformation from EU-weakCMA secure scheme to EU-CMA secure scheme

We review the generic transformation from an EU-weakCMA secure signature scheme to an EU-CMA secure signature scheme using a chameleon hash function [21]. Since this transformation is well-known in crypto community, we review the description of transformation only and omit the detailed proof of security. A chameleon hash function scheme consists of two algorithms CamHSetup and TdColl , which satisfy three properties, namely, collision resistance, trapdoor collisions, and uniformity.

$\text{CamHSetup}(\lambda)$: It takes the security parameter λ as input, and outputs the description of the hash function $H : \mathcal{X} \times \mathcal{R} \rightarrow \mathcal{Y}$ and a trapdoor td .

$\text{TdColl}(td, x, r, x')$: It takes the trapdoor td and inputs of the hash functions $x, x' \in \mathcal{X}$ and $r \in \mathcal{R}$ as inputs. The algorithm then outputs $r' \in \mathcal{R}$.

We require a *secure* chameleon hash function H that satisfies the following three properties, namely collision resistance, trapdoor opening, and uniformity.

- Collision resistance: For any PPT algorithm, it is infeasible to find $(x, r) \neq (x', r') \in \mathcal{X} \times \mathcal{R}$ such that $H(x; r) = H(x'; r')$.
- Uniformity: For each $x \in \mathcal{X}$, $H(x; r)$ has the same probability distribution, where $r \xleftarrow{\$} \mathcal{R}$.
- Trapdoor-Opening: Given $td, x, x' \in \mathcal{X}$, and $r \in \mathcal{R}$, it is possible to efficiently find r' such that $H(x; r) = H(x'; r')$.

Let $\mathcal{S}_{\text{weak}} = (\text{Setup}_w, \text{Sign}_w, \text{Verify}_w)$ be an EU-weakCMA secure signature scheme, and $\mathcal{C} = (\text{CamHSetup}, \text{TdColl})$ be a chameleon hash function scheme. The following scheme is thus an EU-CMA secure signature scheme.

$\text{Setup}(\lambda)$: Run $\text{CamHSetup}(\lambda) \rightarrow (H(\cdot; \cdot), td)$ and $\text{Setup}_w(\lambda) \rightarrow (vk', sk')$, publish $vk = (vk', H)$, and then keep $sk = sk'$.

$\text{Sign}(M, sk)$: Pick a random $r \in \mathcal{R}$, compute $y = H(M; r)$, run $\text{Sign}_w(vk', y, sk') \rightarrow \sigma'$, and then output the signature $\sigma = (\sigma', r)$.

Verify(vk, M, σ): Parse σ as (σ', r) , compute $y = H(M; r)$, and then output $\text{Verify}_w(vk', y, \sigma')$.

Lemma 1 ([19]). If S_{weak} is EU-weakCMA secure and C is a secure chameleon hash function scheme, then the above scheme is an EU-CMA secure signature scheme.

2.4. Background on assumption and mathematics

In this subsection, we review some background on the RSA assumption and some related mathematics. Let $\text{rsa}(\lambda)$ be a polynomially bounded function that maps λ to the bit length of the RSA modulus including useful lemmas, which were occasionally used in the previous works. Note that in this paper, we need the RSA assumption with *safe primes*.

Definition 2 (RSA [25]). Let N be the product of two distinct safe primes p and q with $2^{\frac{\text{rsa}(\lambda)}{2}} \leq p, q \leq 2^{\frac{\text{rsa}(\lambda)}{2}+1} - 1$. That is, both $(p-1)/2$ and $(q-1)/2$ are also primes. Let e be a randomly chosen positive integer less than and relatively prime to $\phi(N) = (p-1)(q-1)$. Let $y \xleftarrow{\$} \mathbb{Z}_N^*$. We state that the RSA assumption holds on N if, for every PPT algorithm $\mathcal{A}$, the following probability is negligible in λ , where the probability goes over the choice of e, y , and the random tape of $\mathcal{A}$:

$$\Pr[\mathcal{A}(N, e, y) \rightarrow x \text{ such that } x^e \equiv y \pmod{N}]$$

In many literatures, the following trick proposed by Shamir is widely used and we also employ it in our security proof.

Lemma 2 (Shamir [28]). Given $x, y \in \mathbb{Z}_N$ together with $a, b \in \mathbb{Z}$ such that $x^a = y^b$ and $\gcd(a, b) = 1$, there is an efficient algorithm for computing $w \in \mathbb{Z}_N$ such that $w^a = y$.

We use some lemmas regarding a prime distribution that are variants of the prime number theorem. Note that prime number theorem is broadly used for the proof of RSA signature schemes, for example [4,16,18,19]. We recall two useful lemmas from [4].

Lemma 3 (Lemma 5.3 in [4]). We first define $\pi(x)$ as the number of primes less than or equal to x . For a positive integer n larger than or equal to 2^{21} , we have

$$\frac{n}{\log n} \leq \pi(n) \leq \frac{2n}{\log n}.$$

Lemma 4 (Lemma 5.5 [4]). Let $\mathcal{P}_k$ be the set of all primes p contained in an interval $[2^{\frac{k}{2}}, 2^k]$. For positive integers ℓ and k , we have that, for all sets $S \subset [1, 2^k]$ with $|S| \leq 2^\ell$,

$$\Pr_{p \xleftarrow{\$} \mathcal{P}_k} [\exists x \in S \text{ such that } p|x] \leq \frac{10k}{2^{k-\ell}}.$$

We also recall the generalized birthday lemma from [16,27], which will be essentially used to analyze the security of the proposed scheme.

Lemma 5 (Generalized Birthday Lemma [16,27]). Let $\mathcal{T}$ be an arbitrary set of size Q , $\mathbf{t}_j$ be a k -dimensional vector in $\mathcal{T}^k$, and $\mathbf{t}_j^{(i)}$ be its i -th prefix. We say that an event E occurs if for randomly chosen q vectors $\mathbf{t}_1, \dots, \mathbf{t}_q \xleftarrow{\$} \mathcal{T}^k$, there are at least $m+1$ distinct indices $j_1, \dots, j_{m+1} \in [1, q]$ such that $\mathbf{t}_{j_1}^{(i)} = \dots = \mathbf{t}_{j_{m+1}}^{(i)}$. Then, we have the following bound.

$$\Pr_{\mathbf{t}_1, \dots, \mathbf{t}_q \xleftarrow{\$} \mathcal{T}^k} [E] < \frac{q^{m+1}}{(m+1)!Q^{im}}.$$

3. New RSA-based signature scheme

In this section, we propose our main signature scheme. Basically, we follow the seminal papers of Hohenberger and Waters [18,19]. Their approach is essential for standard model RSA signatures; that is, we use a pseudorandom function for uniform prime-number generator. Furthermore, we adopt a tag-based approach [4]. That is, each signature is determined by not only a message but also a random tag vector. We use this technique to reduce the number of required prime-numbers in signing algorithm.

In [4], the authors proposed a novel proof technique called “confined-guessing,” and it is mainly used to achieve $O(\log \lambda)$ complexity in terms of the required prime numbers in signing. Although confined-guessing is very powerful technique, there are two potential weak points. One is that confined-guessing is a kind of trade-off between efficiency and reduction loss in the security proof. That is, as the authors said, their improvement is achieved on the basis of loose security reduction. Furthermore, it seems not easy to overcome the barrier of $O(\log \lambda)$ complexity because the confined-guessing strategy is to concatenate $\log \lambda$ schemes and then to optimize the resulting scheme.

In contrast to [4], we use tag vectors with consistent length components, on which we cannot apply confined-guessing strategy, but we can employ the generalized birthday lemma [16,27]. We describe our signature scheme.

Setup (λ): This algorithm chooses an $rsa(\lambda)$ -bit RSA modulus $N = pq$ with safe primes p and q , that is, $p = 2p' + 1$ and $q = 2q' + 1$ for primes p' and q' . It also chooses a random key K for the PRF function $F_K: \{0, 1\}^* \rightarrow \{0, 1\}^{rsa(\lambda)}$, a random $c \in \{0, 1\}^{rsa(\lambda)}$, and random integers $u_0, u_1 \xleftarrow{\$} \mathbb{Z}_N^*$. In addition, K and c are used to establish a function $H_{K,c}: \{0, 1\}^* \rightarrow \{0, 1\}^{rsa(\lambda)}$ defined as $z \mapsto F_K(z \| \mu_z) \oplus c$, where μ_z is the smallest positive integer such that $F_K(z \| \mu_z) \oplus c$ is an odd prime. Then, the verification key vk is (N, u_0, u_1, K, c) . The secret key is the factorization information of N together with all values in vk .

Sign (M, sk): Without a loss of generality, we assume that M is an ℓ -bit message because we can first apply a collision-resistant hash function for larger messages, where $\lambda \leq \ell < rsa(\lambda)/2$. Choose a k -dimensional tag vector $t \xleftarrow{\$} [0, \lambda - 1]^k$ for another parameter $k = \omega(1)$ (e.g., $k = \log \log \lambda$). Let $t^{(i)}$ be an i -dimensional vector that is equal to the i th prefix of t . For $i \in [1, k]$, compute $e_i := H_{K,c}(t^{(i)})$. If e_i divides $\phi(N)$, then the signing algorithm outputs sk . Otherwise, the signing algorithm outputs the following signature σ together with the random tag vector t :

$$\sigma := (u_0 u_1^M)^{\prod_{i=1}^k e_i^{-1}} \bmod N \quad (1)$$

Verify ($vk, M, (\sigma, t)$): The verification algorithm outputs accept if and only if

$$\sigma^{\prod_{i=1}^k H_{K,c}(t^{(i)})} \equiv u_0 u_1^M \bmod N. \quad (2)$$

Correctness. Since Eqs. (1) and (2) are equivalent, the above scheme is correct for valid signatures. More precisely, a valid signature for message M is of the form in Eq. (1) and if we substitute σ in Eq. (2) with that in Eq. (1), then we obtain the following equation.

$$\left((u_0 u_1^M)^{\prod_{i=1}^k (H_{K,c}(t^{(i)}))^{-1}} \right)^{\prod_{i=1}^k H_{K,c}(t^{(i)})} \equiv u_0 u_1^M \bmod N$$

Thus, **Verify** algorithm outputs accept if σ is a valid signature for message M . In fact, if e_i divides $\phi(N)$, then the signature should be sk , so that we can add a pre-processing into the **Verify** algorithm to check whether σ is sk or not. Note that the **Setup** algorithm chose p and q to be safe primes so that all e_i 's do not divide $\phi(N)$, except for negligible probability.

In the above scheme, we set $k = \omega(1)$; for example, $k = \log \log \lambda$, such that each signer has to perform k prime-number generations for a single signing.

4. Proof of security

We analyze the security of the proposed scheme under the RSA assumption in the standard model. As we aforementioned, we first prove its EU-weakCMA security and then apply the generic transform to obtain EU-CMA secure signature scheme.

Theorem 1 (EU-weakCMA Security). *If the RSA assumption holds for N chosen according to the Setup algorithm in the above scheme, then the above scheme is EU-weakCMA secure.*

Proof. Suppose that there exists a PPT algorithm $\mathcal{A}$ that takes vk of the proposed signature scheme as input, and outputs a target message M^* and its forgery (σ^*, t^*) with ε success probability. In particular, we assume that ε is non-negligible in λ . Then, we show that this is a contradiction to the RSA assumption. To this end, we will design a simulator $\mathcal{B}$ that takes an RSA challenge (N, e^*, y) as input, runs $\mathcal{A}$ as its sub-algorithm, and finally outputs a solution x such that $x^{e^*} = y \bmod N$. We first describe $\mathcal{B}$, and then show that its success probability is non-negligible.

Initialization: $\mathcal{B}$ begins by receiving (N, e^*, y) from the RSA problem challenger. If e^* is not prime with $\log(e^*) \geq rsa(\lambda)/2$, then $\mathcal{B}$ aborts. $\mathcal{B}$ runs $\mathcal{A}$, and then receives a list of signing queries. That is, $\mathcal{A}$ sends a set of messages $\{M_1, \dots, M_q \in \mathcal{M}\}$. $\mathcal{B}$ chooses k -dimensional tag vectors $t_1, \dots, t_q \xleftarrow{\$} [0, \lambda - 1]^k$ in advance, where t_i will be used for the i -th signing query. If $t_i = t_j$ for some distinct indices i and $j \in [1, q]$, then $\mathcal{B}$ aborts. Otherwise, we know that all tag vectors are distinct, and in this case $\mathcal{B}$ guesses a prefix of t^* as follows: Let i^* be the smallest index such that $t^{*(i^*)}$ is equal to at most one prefix $t_j^{(i^*)}$ for some $j \in [1, q]$. Next, $\mathcal{B}$ first guesses i^* by choosing it from $[1, k]$ at random, and then guesses $t^{*(i^*)}$. To guess $t^{*(i^*)}$, $\mathcal{B}$ first randomly chooses $t_j^{(i^*-1)}$ from the $(i^* - 1)$ th prefixes of t_j , and then chooses the i^* -th component of $t^{*(i^*)}$ from $[0, \lambda - 1]$ at random. During the remaining description of $\mathcal{B}$, let us assume that the guess of $\mathcal{B}$ is correct.

Setup: $\mathcal{B}$ randomly picks a PRF key K , chooses $\mu^* := \mu_{t^{*(i^*)}} \xleftarrow{\$} [1, rsa(\lambda)^2]$, and computes $c := F_K(t^{*(i^*)} \| \mu^*) \oplus e^*$. For any prefix t of $t_1, \dots, t_q$, if $H_{K,c}(t) = e^*$ and $t \neq t^{*(i^*)}$, then $\mathcal{B}$ aborts. If $H_{K,c}(t^{*(i^*)}) \neq e^*$, then $\mathcal{B}$ also aborts.

If exists, let j^* be the index such that $t_{j^*}^{(i^*)} = t^{*(i^*)}$. (Based on the definition of i^* , we know that such j^* can exist at most one time, or does not exist at all.) In this case, we define $f(X) := X - M_{j^*}$. Otherwise (that is, there is no j^*), we then define $f(X) = X$. For notational convenience, α_0 and α_1 denote the coefficients of $f(X)$, that is, $f(X) = \alpha_0 X + \alpha_1$. We also define another polynomial $g(X) := \beta_0 X + \beta_1$, where $\beta_j \xleftarrow{\$} [0, \lfloor N/4 \rfloor]$. For the sake of simplicity, we use certain notations, namely, $\pi := \prod_{\substack{i \in [1, k] \\ j \in [1, q]}} H_{K,c}(t_j^{(i)})$ and $\pi_I := \frac{\pi}{\prod_{(i,j) \in I} H_{K,c}(t_j^{(i)})}$ if I is well defined. If I is not well defined (e.g., $I = \{i^*\} \times \{j^*\}$) but j^* does not exist), then we define π_I through π .

$\mathcal{B}$ computes $u_i := (y^{\alpha_i \pi_{\{i^*\} \times \{j^*\}} + \beta_i \pi})^2$. We then note that, for any message M ,

$$u_0 u_1^M = (y^{f(M) \pi_{\{i^*\} \times \{j^*\}} + g(M) \pi})^2. \quad (3)$$

Finally, $\mathcal{B}$ sends $vk = \{N, u_0, u_1, K, c\}$ to $\mathcal{A}$.

Sign: For each signing query M_j , $\mathcal{B}$ prepares the corresponding tag vector t_j during the **Setup** phase. We describe how $\mathcal{B}$ computes the corresponding signature σ_j .

For M_{j^*} , $\mathcal{B}$ computes and sends $\mathcal{A}$

$$(y^{g(M_{j^*}) \pi_{[0, \lambda-1] \times \{j^*\}}})^2 \bmod N.$$

For other messages M_j , such that $j \neq j^*$, let J be the union of $[0, \lambda-1] \times \{j\}$ and $\{i^*\} \times \{j^*\}$. Here, $\mathcal{B}$ computes and sends $\mathcal{A}$

$$(y^{f(M_j) \pi_J + g(M_j) \pi_{[0, \lambda-1] \times \{j\}}})^2 \bmod N.$$

Extract from forgery: At the end of an interaction, $\mathcal{A}$ returns $(M^*, (\sigma^*, t^*))$. If the output of $\mathcal{A}$ is not a valid signature, $\mathcal{B}$ aborts. Furthermore, if the guess of $\mathcal{B}$ for the target tag $t^{*(i^*)}$ is wrong, then $\mathcal{B}$ also aborts. Otherwise, the forgery should pass the verification process, that is, it holds that

$$\begin{aligned} & (\sigma^*)^{\prod_{i=1}^k e_i} \\ & \equiv u_0 u_1^{M^*} \equiv (y^{f(M^*) \pi_{\{i^*\} \times \{j^*\}} + g(M^*) \pi})^2 \bmod N, \end{aligned} \quad (4)$$

where $e_i = H_{K,c}(t^{*(i)})$. If $\gcd(e_{i^*}, f(M^*)) > 1$ or $\gcd(e_{i^*}, f(M^*) + g(M^*)) > 1$, then $\mathcal{B}$ aborts. Otherwise, $\mathcal{B}$ diverges according to the existence of j^* .

If j^* exists (that is, $t_{j^*}^{(i^*)} = t^{*(i^*)}$), then we obtain the following equation from Eq. (4).

$$\left(\frac{(\sigma^*)^{\prod_{i \neq i^*} e_i}}{y^{2g(M^*) \pi_{\{i^*\} \times \{j^*\}}}} \right) e_{i^*} \equiv y^{2f(M^*) \pi_{\{i^*\} \times \{j^*\}}} \bmod N \quad (5)$$

Because $\gcd(e_{i^*}, 2f(M^*) \pi_{\{i^*\} \times \{j^*\}}) = 1$ at this point, we can apply Lemma 2 to Eq. (5), allowing $\mathcal{B}$ to compute a solution w such that $w^{e_{i^*}} \equiv y \bmod N$.

Next, let us consider the case in which j^* does not exist (that is, for all $j \in [1, q]$, $t_{j^*}^{(i^*)} \neq t^{*(i^*)}$). In this case, we know that $\pi_{\{i^*\} \times \{j^*\}} = \pi$ and $\gcd(e_{i^*}, \pi) = 1$ (Otherwise, $\mathcal{B}$ is already aborted during the Setup phase.) Therefore, we have $\gcd(e_{i^*}, 2(f(M^*) + g(M^*)) \pi) = 1$, and thus we can apply Lemma 2 to the following equation.

$$((\sigma^*)^{\prod_{i \neq i^*} e_i}) e_{i^*} \equiv y^{2(f(M^*) + g(M^*)) \pi} \bmod N \quad (6)$$

Then, $\mathcal{B}$ can compute a solution w such that $w^{e_{i^*}} \equiv y \bmod N$.

Finally, $\mathcal{B}$ outputs w as its final result, that is, a solution to the RSA challenge (N, e^*, y) .

Analysis: We define a series of games $\text{Game}_0, \dots, \text{Game}_7$. Let Succ_i denote an event in which $\mathcal{A}$ outputs a valid forgery in Game_i . Game_0 is an original EU-weakCMA experiment. Then, through our assumption, we know that $\Pr[\text{Succ}_0] = \text{Adv}_{S, \mathcal{A}}^{\text{EU-weakCMA}}(\lambda) \stackrel{\text{let}}{=} \varepsilon$ is non-negligible. Here, Game_7 is the final game, which is exactly same as the simulator described above. A goal of this analysis is to show the relations between adjacent games, and we finally obtain a lower bound of $\Pr[\text{Succ}_7]$, which is of the form $\varepsilon / \text{poly}(\lambda) - \text{neg}(\lambda)$, where $\text{poly}(\lambda)$ is a polynomial function in λ , and $\text{neg}(\lambda)$ is a negligible function in λ . If ε is non-negligible, then $\Pr[\text{Succ}_7]$ is also non-negligible, and thus the above simulator $\mathcal{B}$ successfully solves the RSA challenge with at least a non-negligible $\Pr[\text{Succ}_7]$ probability.

The simulator's behavior is changed step-by-step in a series of games, and in the final game, the simulator is exactly the same as the above description of our simulator. We will describe such changed parts only. That is, we assume that other parts of the simulator in Game_i are equivalent to Game_{i-1} , except for those mentioned.

- **Game₁:** In this experiment, the simulator aborts if the RSA challenge e^* is not a prime with condition $\log(e^*) \geq \frac{\text{rsa}(\lambda)}{2}$.
- **Game₂:** As in our simulation, $\mathcal{B}$ chooses random tag vectors $t_1, \dots, t_q$ in advance, where t_i will be used for the i -th signing query. If there exists a collision among t_j 's, then $\mathcal{B}$ aborts.
- **Game₃:** The simulator in Game_2 guesses t^* as in our simulator description. If its guess is wrong, $\mathcal{B}$ aborts.
- **Game₄:** $\mathcal{B}$ chooses a PRF secret key K and an index μ^* at random and sets up the function $H_{K,c}$ as in our simulator description. If $H_{K,c}(t^{*(i^*)}) \neq e^*$, then $\mathcal{B}$ aborts.
- **Game₅:** For any prefix t of $t_1, \dots, t_q$, if $H_{K,c}(t) = e^*$ and $t \neq t^{*(i^*)}$, then $\mathcal{B}$ aborts.
- **Game₆:** $\mathcal{B}$ generates part of verification key u_0 and u_1 according to our description of the simulator.
- **Game₇:** If $\gcd(e_{i^*}, f(M^*)) > 1$ or $\gcd(e_{i^*}, f(M^*) + g(M^*)) > 1$, then $\mathcal{B}$ aborts.

Next, we show the relations between $\Pr[\text{Succ}_{i-1}]$ and $\Pr[\text{Succ}_i]$ for $i \in [1, 7]$. The proofs of the following lemmas are relegated to Section 4.1.

Lemma 6 (Game_0 and Game_1). If $\Pr[\text{Succ}_0]$ is non-negligible, $\Pr[\text{Succ}_1]$ is also non-negligible. More precisely, $\Pr[\text{Succ}_1] \geq \frac{1}{8(\text{rsa}(\lambda)+2)} \cdot \Pr[\text{Succ}_0]$

Lemma 7 (Game₁ and Game₂). If $\Pr[\text{Succ}_1]$ is non-negligible, $\Pr[\text{Succ}_2]$ is also non-negligible. More precisely, $\Pr[\text{Succ}_2] > \Pr[\text{Succ}_1] - \frac{q^2}{2\lambda^k}$.

Lemma 8 (Game₂ and Game₃). If $\Pr[\text{Succ}_2]$ is non-negligible, $\Pr[\text{Succ}_3]$ is also non-negligible. More precisely, $\Pr[\text{Succ}_3] = \frac{1}{kq\lambda} \Pr[\text{Succ}_2]$.

Lemma 9 (Game₃ and Game₄). If $\Pr[\text{Succ}_3]$ is non-negligible, $\Pr[\text{Succ}_4]$ is also non-negligible. More precisely, $\Pr[\text{Succ}_4] \geq \frac{1}{\text{rsa}(\lambda)^2} (\Pr[\text{Succ}_3] - \varepsilon_{\text{PRF}} - (\frac{1}{e})^{\text{rsa}(\lambda)})$.

Lemma 10 (Game₄ and Game₅). If $\Pr[\text{Succ}_4]$ is non-negligible, $\Pr[\text{Succ}_5]$ is also non-negligible. More precisely, $\Pr[\text{Succ}_5] \geq \Pr[\text{Succ}_4] - (\varepsilon_{\text{PRF}} + \frac{\text{rsa}(\lambda)qk}{2\text{rsa}(\lambda)})$.

Lemma 11 (Game₅ and Game₆). If $\Pr[\text{Succ}_5]$ is non-negligible, $\Pr[\text{Succ}_6]$ is also non-negligible. More precisely, $\Pr[\text{Succ}_6] \geq \Pr[\text{Succ}_5] - 2^{-\frac{\text{rsa}(\lambda)}{2} + 1}$.

Lemma 12 (Game₆ and Game₇). If $\Pr[\text{Succ}_6]$ is non-negligible, $\Pr[\text{Succ}_7]$ is also non-negligible. More precisely, $\Pr[\text{Succ}_7] \geq \Pr[\text{Succ}_6] - \frac{10\text{rsa}(\lambda)}{2\text{rsa}(\lambda)/2}$.

From the above series of lemmas and our assumption that $\Pr[\text{Succ}_0]$ is non-negligible, we obtain that $\Pr[\text{Succ}_7]$ is non-negligible. Because the extraction strategy of $\mathcal{B}$ succeeds whenever $\mathcal{A}$ outputs a correct forgery, we conclude that $\mathcal{B}$ successfully outputs a solution to an RSA challenge with a non-negligible probability. Therefore, if the RSA assumption holds, it is proved that the proposed scheme is EU-weakCMA secure. $\square$

Theorem 2 (EU-CMA Security). If the RSA assumption holds on N chosen according to the Setup algorithm in the above scheme, we have an EU-CMA secure signature scheme through the transformation given in Section 2.

Proof. From Lemma 1 and Theorem 1, we directly obtain the proof of this.

Remark. Recall the useful trade-off between the signature size and signing/verifying costs, which is broadly used not only in RSA-based signature schemes [4,16,18,19,29], but also in Computational Diffie–Hellman based signature schemes [27]. A random tag vector $\mathbf{t}$ is assigned to each signature with message M . We can set $\mathbf{t}$ as an output of PRF by taking M as an input. This process reduces the size of the signatures because we can remove tag vectors from the signatures. If the signer publishes the PRF key as a part of the verification key, we can still verify the signature without a tag vector because the verifier can always re-generate a tag vector $\mathbf{t}$ from M and vk , and then run the original verifying algorithm. We note that publishing the PRF key is not harmful for a security proof since the uniformity of tag vectors is enough for using the generalized birthday lemma in our security proof. In other words, in the EU-weakCMA security model, the simulator use the PRF key to generate tag vectors with uniform distribution such that there is no collisions among tag vectors with overwhelming probability due to the generalized birthday lemma, and then publish the PRF key. After publishing the PRF key, we can no longer expect indistinguishability between tag vectors and random vectors, but we have already achieved our purpose (no collisions among tag vectors). In the EU-CMA security model, the adversary can query after receiving the PRF key, but we use the transformation using chameleon hashes such that the input of the pseudorandom function is already random, which is the output of the chameleon hash and independent from the message query. Therefore, one can easily adapt this well-known trade-off technique to our RSA signature scheme.

4.1. Proof of Lemmas

In this subsection, we provide all missing proofs.

Proof of Lemma 6. Let $\mathcal{P}_k$ be a set of primes p with $2^{\frac{k}{2}} \leq p \leq 2^k$. In Game₁, the simulator runs without aborting if and only if $e^* \in \mathcal{P}_{\text{rsa}(\lambda)}$. We calculate

$$\begin{aligned}
 & \Pr[e^* \in \mathcal{P}_{\text{rsa}(\lambda)}] \\
 &= \frac{\pi(\phi(N)) - \pi(2^{\text{rsa}(\lambda)/2})}{\phi(\phi(N))} \\
 &\geq \frac{\frac{\phi(N)}{\log \phi(N)} - \frac{2^{\text{rsa}(\lambda)/2+1}}{\text{rsa}(\lambda)}}{\phi(\phi(N))} && // \text{ by Lemma 3} \\
 &\geq \frac{\frac{\phi(N)}{\log \phi(N)} - \frac{2^{\text{rsa}(\lambda)/2+1}}{\text{rsa}(\lambda)}}{2^{\text{rsa}(\lambda)}} && // \phi(\phi(N)) \leq 2^{\text{rsa}(\lambda)} \\
 &\geq \frac{\frac{N/4}{\log N} - \frac{2^{\text{rsa}(\lambda)/2+1}}{\text{rsa}(\lambda)}}{2^{\text{rsa}(\lambda)}} && // N > \phi(N) \geq N/4 \\
 &> \frac{2^{\text{rsa}(\lambda)-2} - \frac{2^{\text{rsa}(\lambda)/2+1}}{\text{rsa}(\lambda)}}{2^{\text{rsa}(\lambda)}} && // 2^{\text{rsa}(\lambda)+2} > N \geq 2^{\text{rsa}(\lambda)} \\
 &> \frac{1}{8(\text{rsa}(\lambda)+2)}
 \end{aligned}$$

Therefore, we have

$$\Pr[\text{Succ}_1] \geq \frac{1}{8(\text{rsa}(\lambda) + 2)} \cdot \Pr[\text{Succ}_0].$$

The denominator in the right-hand side is a polynomial function in λ , so that if $\Pr[\text{Succ}_0]$ is non-negligible, $\Pr[\text{Succ}_1]$ is also non-negligible. $\square$

Proof of Lemma 7. The simulator guesses the target tag t^* in advance, which is expected to be used by $\mathcal{A}$ as a forgery. Using Lemma 5, we can calculate the upper bound of the collision probability among t_j 's as follows.

$$\Pr_{t_1, \dots, t_q \xleftarrow{\$} [0, \lambda-1]^k} [\exists j_1 \neq j_2 \in [1, q] \text{ such that } t_{j_1} = t_{j_2}] < \frac{q^2}{2\lambda^k}$$

Therefore, we have

$$\Pr[\text{Succ}_2] > \Pr[\text{Succ}_1] - \frac{q^2}{2\lambda^k}.$$

When $k = \omega(1)$ and q is an arbitrary polynomially bounded function, it is clear that $\frac{q^2}{2\lambda^k}$ is a negligible function in λ . Therefore, if $\Pr[\text{Succ}_1]$ is non-negligible, $\Pr[\text{Succ}_2]$ is also non-negligible. $\square$

Proof of Lemma 8. Recall the definition of i^* , which is the smallest index such that $t^{*(i^*)}$ is equal to at most one prefix $t_j^{(i^*)}$ for a certain $j \in [1, q]$. We know that i^* should exist between 1 and q because we remove all other cases from Game₂. Therefore, the guessing of B for i^* is correct with a probability of $1/k$. Through the smallest condition, $t^{*(i^*-1)}$ is equal to $t_j^{(i^*-1)}$ for a certain j . Thus, the guessing of B for such j is correct with a probability of $1/q$. Finally, guessing for the i^* th component should be correct with $1/\lambda$ because its domain size is λ . All such guesses are independent and hidden from an adversary, and thus we have

$$\Pr[\text{Succ}_3] = \frac{1}{kq\lambda} \Pr[\text{Succ}_2].$$

Although we set $k = \omega(1)$, we do not consider large k such as $k = \omega(\lambda)$. $k = \log \log \lambda$ or $k = \log \log \log \lambda$ are typical examples for our parameter selection, and thus $kq\lambda$ is polynomially bounded. Therefore, we conclude that if $\Pr[\text{Succ}_2]$ is non-negligible, $\Pr[\text{Succ}_3]$ is also non-negligible. $\square$

Proof of Lemma 9. In Game₄, $\mathcal{B}$ generates K and c according to our simulation description and then aborts if $H_{K,c}(t^{*(i^*)}) \neq e^*$. First, we assume that F_K is a truly random function. Then, for each execution of F_K , its output is an $\text{rsa}(\lambda)$ -bit prime with at least $1/\text{rsa}(\lambda)$ probability through Lemma 3. Thus, if we execute F_K $\text{rsa}(\lambda)^2$ times for mutually distinct inputs, then the probability of not outputting an $\text{rsa}(\lambda)$ -bit prime is at most $(1 - 1/\text{rsa}(\lambda))^{\text{rsa}(\lambda)^2} < (1/e)^{\text{rsa}(\lambda)}$. For a pseudorandom function (instead of truly random function) F_K , we can obtain a similar upper bound $\varepsilon_{\text{PRF}} + (1/e)^{\text{rsa}(\lambda)}$, where ε_{PRF} is the maximum distinguishing advantage of F_K . (We can construct a PRF distinguisher if the output of F_K has a different distribution from a truly random function. This is a common argument used in crypto studies, for example, [4,16,19].) Under the condition that $\text{rsa}(\lambda)^2$ number of F_K outputs contain at least one $\text{rsa}(\lambda)$ -bit prime, $\mathcal{B}$ hits one such prime with $1/\text{rsa}(\lambda)^2$ probability by choosing $\mu^* \xleftarrow{\$} [1, \text{rsa}(\lambda)^2]$. Therefore, we have

$$\Pr[\text{Succ}_4] \geq \frac{1}{\text{rsa}(\lambda)^2} \left(\Pr[\text{Succ}_3] - \varepsilon_{\text{PRF}} - \left(\frac{1}{e} \right)^{\text{rsa}(\lambda)} \right).$$

$\varepsilon_{\text{PRF}} + \left(\frac{1}{e} \right)^{\text{rsa}(\lambda)}$ is negligible in λ under the assumption that F_K is a secure pseudorandom function. Since $\text{rsa}(\lambda)^2$ is a polynomial in λ , we conclude that if $\Pr[\text{Succ}_3]$ is non-negligible, $\Pr[\text{Succ}_4]$ is also non-negligible. $\square$

Proof of Lemma 10. Similar to the previous proof, herein we first assume that F_K is a truly random function. Then, for each prefix t of $t_1, \dots, t_q$, $H_{K,c}(t)$ is an $\text{rsa}(\lambda)$ -bit prime. Through Lemma 3, the probability that e^* will hit a particular $\text{rsa}(\lambda)$ -bit prime is at most $\frac{\text{rsa}(\lambda)}{2\text{rsa}(\lambda)}$. Because we consider at most qk inputs of $H_{K,c}$, the probability that e^* will hit one of the outputs of $H_{K,c}$ with our inputs is at most $\frac{\text{rsa}(\lambda)qk}{2\text{rsa}(\lambda)}$. For a pseudorandom function (instead of a truly random function) F_K , we can obtain a similar upper bound $\varepsilon_{\text{PRF}} + \frac{\text{rsa}(\lambda)qk}{2\text{rsa}(\lambda)}$, and thus we have

$$\Pr[\text{Succ}_5] \geq \Pr[\text{Succ}_4] - \left(\varepsilon_{\text{PRF}} + \frac{\text{rsa}(\lambda)qk}{2\text{rsa}(\lambda)} \right).$$

We can easily check that $(\varepsilon_{\text{PRF}} + \frac{\text{rsa}(\lambda)qk}{2\text{rsa}(\lambda)})$ is a negligible function in λ under the assumption that F_K is a secure pseudorandom function. Therefore, we conclude that if $\Pr[\text{Succ}_4]$ is non-negligible, $\Pr[\text{Succ}_5]$ is also non-negligible. $\square$

Proof of Lemma 11. From the following inequality

$$N/4 - p'q' = (p' + q')/2 + 1/4 < 2^{\frac{\text{rsa}(\lambda)}{2} + 1},$$

Table 1

EU-CMA secure short RSA-based signature schemes in the standard model.

Scheme	PK size	Sig. size	PNG in signing	Sec. model	Proof strategy
HW09 [19]	$O(1)$	1_N	$O(\lambda)$	EU-CMA	Prefix-guessing for messages
BHJKS15 [4]	$O(1)$	1_N^b	$O(\log(\lambda))$	EU-CMA	Confined-guessing
This study	$O(1)$	1_N^b	$f(\lambda)^a$	EU-CMA	Prefix-guessing for tags

‘PNG’ denotes Prime Number Generation.

 1_N is a bit-length to express an element in $\mathbb{Z}_N$.^a $f(\lambda)$ can be any strictly increasing function, that is, $f \in \omega(1)$; e.g., $\log \log \lambda$.^b It does not contain the tag vector size because we can apply a tag compression technique given in **Remark**.

we can calculate that the statistical distance between the uniform distribution on $[0, \lfloor N/4 \rfloor]$ and the uniform distribution on $[0, p'q']$ is less than $2^{-\frac{rsa(\lambda)}{2}+1}$.

In Game₆, β_0 and β_1 are chosen from $[0, \lfloor N/4 \rfloor]$ at random, which are distributed statistically indistinguishably from a uniform distribution on $[0, p'q']$, such that they can blind α_0 and α_1 from the viewpoint of $\mathcal{A}$, with the exception of negligible probability $2^{-\frac{rsa(\lambda)}{2}+1}$. Therefore, u_0 and u_1 are uniformly distributed, except for a probability of at most $2^{-\frac{rsa(\lambda)}{2}+1}$, and thus we have

$$\Pr[\text{Succ}_6] \geq \Pr[\text{Succ}_5] - 2^{-\frac{rsa(\lambda)}{2}+1}.$$

The function $2^{-\frac{rsa(\lambda)}{2}+1}$ is clearly negligible in λ , so that $\Pr[\text{Succ}_6]$ is negligible if $\Pr[\text{Succ}_5]$ is negligible. $\square$

Proof of Lemma 12. First, we trivially have $\gcd(e_{i^*}, f(M^*)) = 1$ because e_{i^*} is a prime larger than $2^{rsa(\lambda)/2} > f(M^*)$. Let us compute the probability that a random prime $e_{i^*} \xleftarrow{\$} \mathcal{P}_{rsa(\lambda)}$ divides $f(M^*) + g(M^*)$. Because $\{f(M^*) + g(M^*)\}_{M^* \in \{0,1\}^\ell}$ is a subset of $[1, 2^{rsa(\lambda)}]$ with a size of 2^ℓ , Lemma 4 implies that

$$\begin{aligned} & \Pr_{e_{i^*} \xleftarrow{\$} \mathcal{P}_{rsa(\lambda)}} [\exists f(M^*) + g(M^*) \text{ such that } e_{i^*} | f(M^*) + g(M^*)] \\ & \leq \frac{10rsa(\lambda)}{2^{rsa(\lambda)-\ell}} < \frac{10rsa(\lambda)}{2^{rsa(\lambda)/2}}. \end{aligned}$$

Therefore, we have

$$\Pr[\text{Succ}_7] \geq \Pr[\text{Succ}_6] - \frac{10rsa(\lambda)}{2^{rsa(\lambda)/2}}.$$

One can easily check that $\frac{10rsa(\lambda)}{2^{rsa(\lambda)/2}}$ is a negligible function in λ . Therefore, we conclude that if $\Pr[\text{Succ}_6]$ is non-negligible, $\Pr[\text{Succ}_7]$ is also non-negligible. $\square$

5. Comparison

The proposed digital signature scheme falls into the category of *stateless, hash-then-sign, (standard) RSA-based signature schemes in the standard model*. In this section, we compare our signature scheme with previous signature schemes in this category. Finding a scheme in this category had been a long-standing open problem before Hohenberger and Waters' seminal work [19]. Their scheme is quite efficient in terms of the public key and signature sizes but requires signers to conduct an expensive prime-number generation $O(\lambda)$ times, where λ is the security parameter. Hohenberger and Waters mainly uses so-called the ‘prefix-guessing’ proof technique. Subsequently, Böhl, Hofheinz, Jäger, Koch, and Striecks improved the Hohenberger-Waters signatures in terms of the number of required prime-number generations in the signing algorithm to be $O(\log \lambda)$ through the use of a new proof technique called ‘confined-guessing.’ In this study, we reduce the required number of prime-number generations to be almost constant, or more precisely, $\omega(1)$ (e.g., $\log \log \lambda$). We provide a short comparison in Table 1.

6. Conclusion

In this paper, we propose an efficient digital signature scheme in terms of costs in signing algorithm. More precisely, we reduce the cost for expensive prime-number generations by reducing the required number of primes in signing algorithm. Although we achieve almost constant complexity for such the required prime numbers in signing algorithm, it is still expensive compared with other RSA signature schemes under stronger assumptions. In 2015, Cash, Dowsley, and Kiltz proposed a RSA-based signature scheme without prime-number generation in the standard model [8]. However, their scheme relies on the strong RSA assumption, and a digital signature scheme from the RSA assumption in the standard model *without prime-number generations* is still an important open problem.

Declaration of Competing Interest

We wish to confirm that there are no known conflicts of interest associated with this publication and there has been no significant financial support for this work that could have influenced its outcome.

Acknowledgement

This work was supported by a research fund from [Hanyang University](#) (HY-2018).

References

- [1] N. Attrapadung, B. Libert, Homomorphic network coding signatures in the standard model, in: PKC 2011, in: LNCS, 6571, Springer, 2011, pp. 17–34.
- [2] M. Bellare, P. Rogaway, Random oracles are practical: a paradigm for designing efficient protocols, in: ACM CCS 1993, 1993, pp. 62–73.
- [3] D.J. Bernstein, D. Hopwood, A. Hülsing, T. Lange, R. Niederhagen, L. Papachristodoulou, M. Schneider, P. Schwabe, Z. Wilcox-O’Hearn, SPHINCS: practical stateless hash-based signatures, in: EUROCRYPT 2015, in: LNCS, 9056, Springer, 2015, pp. 368–397.
- [4] F. Böhl, D. Hofheinz, T. Jager, J. Koch, C. Striecks, Confined guessing: new signatures from standard assumptions, J. Cryptol. 28 (2015) 176–208. Springer
- [5] D. Boneh, D.M. Freeman, Homomorphic signatures for polynomial functions, in: EUROCRYPT 2011, in: LNCS, 6632, Springer, 2011, pp. 149–168.
- [6] D. Boneh, D.M. Freeman, Linearly homomorphic signatures over binary fields and new tools for lattice-based signatures, in: PKC 2011, in: LNCS, 6571, Springer, 2011, pp. 1–16.
- [7] C. Brzuska, H. Busch, O.D.M. Fischlin, M. Franz, S. Katzenbeisser, M. Manulis, C. Onete, A. Peter, B. Poettering, D. Schröder, Redactable signatures for tree-structured data: Definitions and constructions, in: ACNS 2010, in: LNCS, 6123, Springer, 2010, pp. 87–104.
- [8] D. Cash, R. Dowsley, E. Kiltz, Digital signatures from strong RSA without prime generation, in: PKC 2015, in: LNCS, 9020, Springer, 2015, pp. 217–235.
- [9] R. Cramer, I. Damgård, New generation of secure and practical RSA-based signatures, in: CRYPTO 1996, in: LNCS, 1109, Springer, 1996, pp. 173–185.
- [10] R. Cramer, V. Shoup, Signature schemes based on the strong rsa assumption, in: ACM Transactions on Information and Systems Security, 3, 2000, pp. 161–185.
- [11] C. Dwork, M. Naor, An efficient existentially unforgeable signature scheme and its applications, in: CRYPTO 1994, in: LNCS, 839, Springer, 1994, pp. 234–246.
- [12] M. Fischlin, The cramer-shoup strong-rsa signature scheme revisited, in: PKC 2003, in: LNCS, 2567, Springer, 2003, pp. 116–129.
- [13] D.M. Freeman, Improved security for linearly homomorphic signatures: a generic framework, in: PKC 2012, in: LNCS, 7293, Springer, 2012, pp. 697–714.
- [14] R. Gennaro, S. Halevi, T. Rabin, Secure hash-and-sign signatures without the random oracle, in: EUROCRYPT 1999, in: LNCS, 1592, Springer, 1999, pp. 123–139.
- [15] S. Goldwasser, S. Micali, R.L. Rivest, A digital signature scheme secure against adaptive chosen-message attacks, SIAM J. Comput. 17 (1988) 281–308.
- [16] D. Hofheinz, T. Jager, E. Kiltz, Short signatures from weaker assumptions, in: ASIACRYPT 2011, in: LNCS, 7073, Springer, 2011, pp. 647–666.
- [17] D. Hofheinz, E. Kiltz, Programmable hash functions and their applications, J. Cryptol. 25 (2012) 484–527. Springer
- [18] S. Hohenberger, B. Waters, Realizing hash-and-sign signatures under standard assumptions, in: EUROCRYPT 2009, in: LNCS, 5479, Springer, 2009, pp. 333–350.
- [19] S. Hohenberger, B. Waters, Short and stateless signatures from the rsa assumption, in: CRYPTO 2009, in: LNCS, 5677, Springer, 2009, pp. 654–670.
- [20] R. Johnson, D. Molnar, D.X. Song, D. Wagner, Homomorphic signature schemes, in: CT-RSA 2002, in: LNCS, 2271, Springer, 2002, pp. 244–262.
- [21] H. Krawczyk, T. Rabin, Chameleon signatures, NDSS 2000, The Internet Society, 2000.
- [22] L. Lamport, Constructing digital signatures from a one-way function, Technical Report SRI-CSL-98, SRI International Computer Science Laboratory, 1979.
- [23] M. Naor, M. Yung, Universal one-way hash functions and their cryptographic applications, in: ACM STOC, ACM Press, 1989, pp. 33–43.
- [24] R. Nojima, J. Tamura, Y. Kadoyayashi, H. Kikuchi, A storage efficient redactable signature in the standard model, in: ISC 2009, in: LNCS, 5735, Springer, 2009, pp. 326–337.
- [25] R. Rivest, A. Shamir, L. Adleman, A method for obtaining digital signatures and public-key cryptosystems, in: Communications of the ACM, 21, ACM New York, NY, USA, 1978, pp. 120–126.
- [26] J. Rompel, One-way functions are necessary and sufficient for secure signatures, in: 22nd ACM STOC, ACM Press, 1990, pp. 387–394.
- [27] J.H. Seo, Short signature from Diffie-Hellman: realizing almost compact public key, J. Cryptol. 30 (3) (2017) 735–759.
- [28] A. Shamir, On the generation of cryptographically strong pseudorandom sequences, ACM Trans. Comput. Syst. (1983) 38–44.
- [29] S. Yamada, G. Hanaoka, N. Kunihiro, Space efficient signature schemes from the RSA assumption, in: PKC 2012, in: LNCS, 7293, Springer, 2012, pp. 102–119.